

Constructors & the Order Things Get Built

Review worksheet — default/overloaded/copy constructors, initializer lists, `const` members — builds on Worksheet 02

Name _____ Date _____ Section _____

part 1 What Is a Constructor?

A **constructor** is a special member function that initializes an object when the object is created.

```
class Player {
    string name;
    int health;
public:
    Player() {
        name = "Unknown";
        health = 100;
    }
};
```

When we write `Player p;`, C++ automatically calls `Player()`.

QUESTION 1

FILL IN THE BLANKS

What makes a constructor different from a normal member function?

1. A constructor has the same _____ as the class.
2. A constructor has no _____ type.
3. A constructor is normally called when an _____ is created.

part 2 Default Constructor

A **default constructor** is a constructor that can be called **without supplying arguments**. Given the `Player` class above, we can write `Player p;` and the resulting object contains:

```
p
+-----+
| name  "Unknown" |
| health 100      |
+-----+
```

QUESTION 2

FILL IN THE VALUES

```
Player hero;
```

hero.name = _____ hero.health = _____

QUESTION 3

WRITE THE CONSTRUCTOR

Add a default constructor to `Weapon`. A default `Weapon` should have the name `"Fists"` and damage `1`.

```
class Weapon {
    string name;
    int damage;
public:
```

```
};
```

Constructors & the Order Things Get Built

Review worksheet — default/overloaded/copy constructors, initializer lists, `const` members — builds on Worksheet 02

part 3 Overloaded Constructors

Just like normal functions, constructors can be **overloaded** — a class can have multiple constructors as long as their parameter lists differ.

```
class Player {
    string name;
    int health;
public:
    Player() { name = "Unknown"; health = 100; }
    Player(string n) { name = n; health = 100; }
    Player(string n, int h) { name = n; health = h; }
};
```

Now there are three ways to create a `Player`: `Player a;` `Player b("Bob");` `Player c("Alice", 75);`

QUESTION 4

WHICH CONSTRUCTOR RUNS?

STATEMENT	CONSTRUCTOR USED	NAME	HEALTH
<code>Player a;</code>	_____	_____	_____
<code>Player b("Kevin");</code>	_____	_____	_____
<code>Player c("Terry", 9000);</code>	_____	_____	_____

QUESTION 5

EXPLAIN

Why is the following **not** an overload of the existing `Player(string n)` constructor?

```
Player(string playerName) {
    name = playerName;
    health = 50;
}
```

Hint: does C++ care about the parameter **name** or the parameter **type** when distinguishing overloads?

Constructors & the Order Things Get Built

Review worksheet — default/overloaded/copy constructors, initializer lists, `const` members — builds on Worksheet 02

part 4

Copy Constructor

Suppose we already have `Player original("Bob", 80);` and want a new object built from it: `Player clone(original);`. C++ uses a **copy constructor**:

```
Player(const Player& other) {
    name = other.name;
    health = other.health;
}
```

We're passing another `Player` object by **const reference**.

QUESTION 6

FILL IN THE VALUES

Player original("Mario", 95);
Player copy(original);

original		copy	
name	_____	name	_____
health	_____	health	_____

Are `original` and `copy` the same object? _____ Do they initially contain the same values? _____

QUESTION 7 — SPOT THE COPY

MARK COPY / NOT COPY

Assume `Player a("Bob", 100);`

Player b(a); _____

Player c = a; _____

Player d; _____

Player e("Alice"); _____

Constructors & the Order Things Get Built

Review worksheet — default/overloaded/copy constructors, initializer lists, `const` members — builds on Worksheet 02

part 5 Why Copy Constructors Matter

For a class with plain members like `string` / `int`, the compiler-generated copy constructor is fine. But once a class holds a **pointer**...

```
class IntArray {
    int* data;
    int size;
};
```

...copying the pointer instead of what it points to gives two objects pointing at the *same* memory (a **shallow copy**):

```
Array A          Array B
data -----+    data -----+
      |              |
      +----> [10][20][30] <-----+
```

A custom copy constructor can allocate new memory instead — a **deep copy**:

```
Array A          Array B
data ----> [10][20][30]  data ----> [10][20][30]
                    separate memory      separate memory
```

QUESTION 8

[EXPLAIN](#)

Why could shallow copying dynamically allocated memory be dangerous?

part 6 Constructor Initializer Lists

C++ provides a second syntax for constructors — the **initializer list** — which initializes members *before* the constructor body runs:

BODY ASSIGNMENT	INITIALIZER LIST
<pre>Player(string n, int h) { name = n; health = h; }</pre>	<pre>Player(string n, int h) : name(n), health(h) { }</pre>

These often produce the same values, but they don't mean the same thing: *construct members* → *enter body* → *assign new values* vs. *construct members using desired values* → *enter body*. That distinction matters once members are themselves objects.

QUESTION 9 — REWRITE IT

[USE AN INITIALIZER LIST](#)

```
Weapon(string n, int d) {
    name = n;
    damage = d;
}
```

```
Weapon(string n, int d)
{
    _____
}
```

Constructors & the Order Things Get Built

Review worksheet — default/overloaded/copy constructors, initializer lists, `const` members — builds on Worksheet 02

part 7 When Initializer Lists Are Required

Sometimes an initializer list isn't just prettier — it's **required**. Consider a `const` member:

```
class Student {
    const int id;
    string name;
};
```

This will not compile — `id` is supposed to already have a value by the time the body runs:

```
Student(int i, string n) {
    id = i;    // ERROR
    name = n;
}
```

Instead: `Student(int i, string n) : id(i), name(n) {}`. Initializer lists are especially important for `const` data members, reference members, member objects that need constructor arguments, and avoiding unnecessary default construction followed by assignment.

part 8 Put Them Together

```
class Robot {
    string name;
    int battery;
public:
    Robot() : name("Unknown"), battery(100) {}
    Robot(string n) : name(n), battery(100) {}
    Robot(string n, int b) : name(n), battery(b) {}
    Robot(const Robot& other) : name(other.name), battery(other.battery) {}
};
```

QUESTION 10 — IDENTIFY THEM

LABEL EACH CONSTRUCTOR

Robot()

Robot(string n)

Robot(string n, int b)

Robot(const Robot& other)

QUESTION 11 — TRACE THE PROGRAM

FILL IN THE TABLE

```
Robot a;
Robot b("R2D2");
Robot c("Johnny 5", 75);
Robot d(c);
```

OBJECT	CONSTRUCTOR USED	NAME	BATTERY
a			
b			
c			
d			

Constructors & the Order Things Get Built

Review worksheet — default/overloaded/copy constructors, initializer lists, `const` members — builds on Worksheet 02

part 9 Find the Error

```
class Course {
    const int number;
    string name;
public:
    Course() {
        number = 2143;
        name = "OOP";
    }
};
```

QUESTION 12

COMPILES? Y / N

Will this compile? _____ If not, explain why:

Rewrite the constructor correctly:

```
Course()
{
    _____
}
```

part 10 Write the Class

Book

Create a class `Book` with `string title;`, `string author;`, `int pages;`. Implement: (1) a default constructor — title/author "Unknown", pages 0; (2) an overload taking title + author; (3) an overload taking title + author + pages; (4) a copy constructor. Use initializer lists for every constructor.

```
class Book {
```

```
};
```

Constructors & the Order Things Get Built

Review worksheet — default/overloaded/copy constructors, initializer lists, `const` members — builds on Worksheet 02

part 11

Predict Before Running

using your Book class

```
Book a;
Book b("Dune", "Frank Herbert");
Book c("The Hobbit", "J.R.R. Tolkien", 310);
Book d(c);
```

a

title

author

pages

b

title

author

pages

c

title

author

pages

d

title

author

pages

part 12

Constructor Challenge

```
class Thing {
    int x;
public:
    Thing() : x(0) { cout << "A\n"; }
    Thing(int value) : x(value) { cout << "B\n"; }
    Thing(const Thing& other) : x(other.x) { cout << "C\n"; }
};
```

WHAT IS PRINTED?

PREDICT, THEN COMPILE AND CHECK

```
Thing a;
Thing b(10);
Thing c(b);
Thing d = c;
```

Prediction: _____

Was your prediction correct? _____ If not, what surprised you?

cheat sheet

Constructors at a Glance

CONCEPT	TYPICAL SYNTAX	PURPOSE
Default constructor	<code>Player()</code>	Create object without arguments
Overloaded constructor	<code>Player(string n)</code>	Create object using supplied values
Copy constructor	<code>Player(const Player& p)</code>	Create a new object from another object
Initializer list	<code>Player(...) : name(n), health(h)</code>	Initialize members directly, before the body runs

An initializer list is **not** a fourth kind of constructor — `Player()`, `Player(string n)`, and `Player(const Player& p)` describe *which* constructor runs; `: name(n), health(100)` describes *how* that constructor initializes its members.